

lab_wine_soln

April 22, 2019

1 Lab: Hyper-Parameter Optimization with PCA

PCA is often applied as a pre-processing step with classifiers. When using PCA in this manner, one must select the number of PC components to use along with parameters in classifier. In this lab, we will demonstrate how to performing this *hyper-parameter optimization*. In doing the lab, you will learn to:

- Combine PCA with data scaling.
- Compute and visualize PC components
- Select the number of PCs with K-fold cross validation
- Implement the multi-stage classifier pipeline in sklearn
- Perform automatic parameter search using GridSearchCV in combination with a pipeline.

We first download the basic packages.

```
In [1]: import numpy as np
import matplotlib
import matplotlib.pyplot as plt
```

1.1 Downloading the Data

We will use a very simple wine dataset, commonly used in teaching machine learning class. The problem is to classify the type of red wine from features of the wine such as the alcohol and other chemical components. There are three possible wine types.

```
In [2]: from sklearn.datasets import load_wine
from sklearn.model_selection import KFold
data = load_wine()

# TODO print the features names in data.feature_names and data.target_names
print(data.feature_names)
print(data.target_names)

['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids',
'class_0' 'class_1' 'class_2']
```

Get the data matrix X from `data.data` and the target values y from `data.target`. Print the number of samples, number of features and number of classes.

```

In [3]: # TODO
        #     X = ...
        #     y = ...
        X = data.data
        y = data.target

        nsamp, nfeatures = X.shape
        nclass = np.max(y) + 1
        print('num samples    = %d' % nsamp)
        print('num features   = %d' % nfeatures)
        print('num classeses  = %d' % nclass)

num samples    = 178
num features   = 13
num classeses  = 3

```

1.2 Perform PCA for Visualization

Before performing PCA, you should scale the data matrix to remove the mean and normalize the variance of the different components. For this purpose, create a `StandardScaler` object scaling. Then fit the scaling with the entire data `X`. Transform the data and let `Xs` be the scaled data.

```

In [4]: from sklearn.preprocessing import StandardScaler

        # TODO
        # scaling = ...
        # scaling.fit(...)
        # Xs = ...
        scaling = StandardScaler()
        scaling.fit(X)
        Xs = scaling.transform(X)

```

Now, fit a PCA on the scaled data matrix `Xs`. You can use the `sklearn` PCA method. In order that we can visualize the results set `n_components=2`. Select `svd_solver='randomized'` and `whiten=True`. Use the `pca.transform` method to find, `Z`, the coefficients of `Xs` in the PCA basis.

```

In [5]: from sklearn.decomposition import PCA

        # TODO
        # ncomp = 2
        # pca = PCA(...)
        # Z = ...

        # Construct the PCA object
        ncomp = 2
        pca = PCA(n_components=ncomp,
                  svd_solver='randomized', whiten=True)

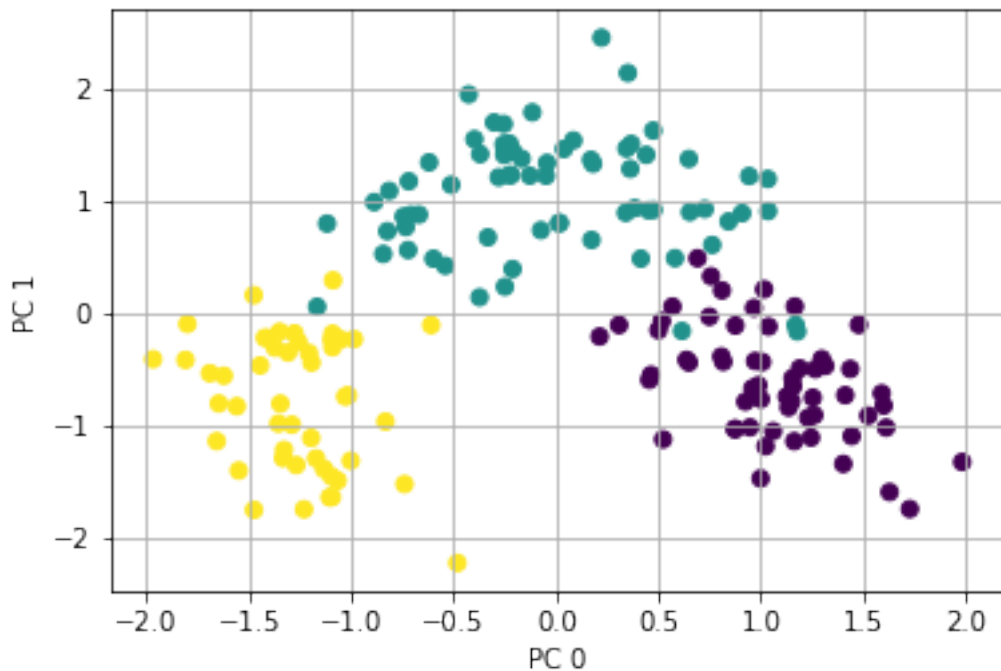
```

```
pca.fit(Xs)
Z = pca.transform(Xs)
```

In the transformed basis, each data sample is represented by a two dimensional vector, $Z[i,0]$, $Z[i,1]$. Plot a scatter plot of the transformed data. Use different marker colors for the different classes. If you did everything, you should see that the classes are quite well separated with even two PCs.

```
In [6]: # TODO
plt.scatter(Z[:,0], Z[:,1], c=y)
plt.grid()
plt.xlabel('PC 0')
plt.ylabel('PC 1')
```

```
Out[6]: Text(0, 0.5, 'PC 1')
```

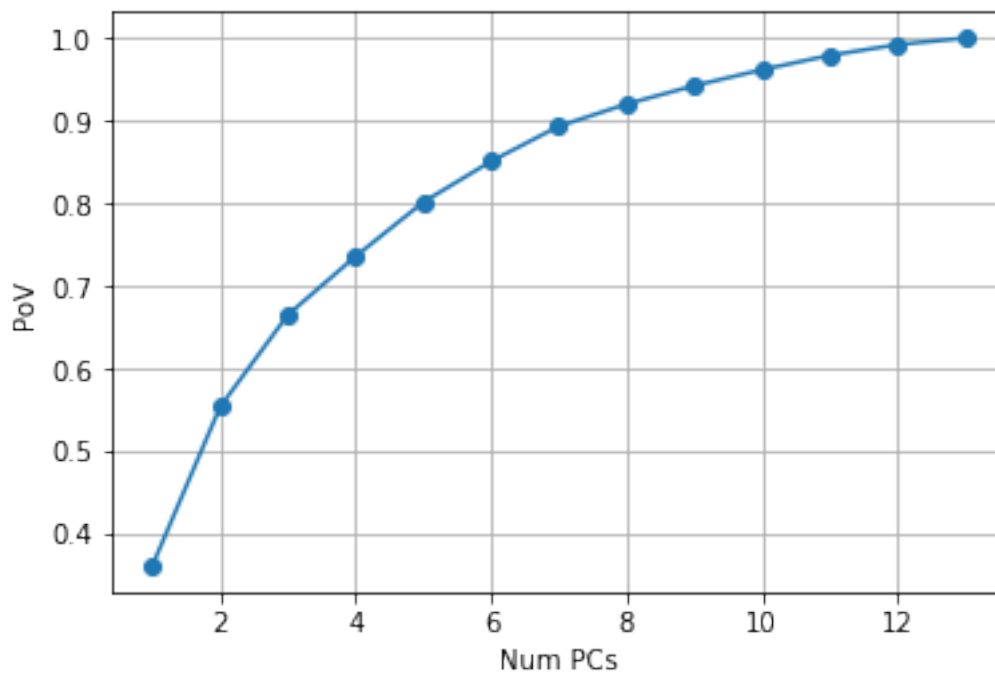


Now, refit the scaled data Xs using $n_components=nfeatures$ where $nfeatures$ is the number of features. This is the maximum number of PCs. Get the singular values from $pca.singular_values_$ and plot the portion of variation as a function of the number of PCs. The PoV for using n PCs is:

$$PoV[n] = \frac{\sum_{i=0}^{n-1} s[i]**2}{\sum_{i=0}^{d-1} s[i]**2}$$

where $s[i]$ is the i -th singular value and d is the number of features. You should see that the 4 PCs contains more than 70% of the variance.

```
In [7]: # TODO
pca = PCA(svd_solver='randomized', whiten=True)
pca.fit(Xs)
s = pca.singular_values_**2
pov = np.cumsum(s)/np.sum(s)
numpc = np.arange(1,nfeatures+1)
plt.plot(numpc,pov, 'o-')
plt.xlabel('Num PCs')
plt.ylabel('PoV')
plt.grid()
```



1.3 Using PCA with Classification

We will now use data scaling and PCA as a pre-processing step for logistic classification. The number of PCs to use can be found with cross-validation. Complete the code below which tries different number of PCs components to use and measures the test accuracy for each value.

```
In [8]: from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
nfold = 5

# Create a K-fold object
kf = KFold(n_splits=nfold)
kf.get_n_splits(X)
```

```

# Number of PCs to try
ncomp_test = np.arange(2,12)
num_nc = len(ncomp_test)

# Accuracy: acc[icomp,ifold] is test accuracy when using `ncomp = ncomp_test[icomp]`
acc = np.zeros((num_nc,nfold))

# Loop over number of components to test
for icomp, ncomp in enumerate(ncomp_test):

    # Look over the folds
    for ifold, I in enumerate(kf.split(X)):
        Itr, Its = I

        # TODO: Split data into training
        # Xtr, Xts, ytr, yts = ...
        Xtr = X[Itr]
        Xts = X[Its]
        ytr = y[Itr]
        yts = y[Its]

        # TODO: Create a scaling object and fit the scaling on the training data
        scaling = StandardScaler()
        scaling.fit(Xtr)
        Xtrs = scaling.transform(Xtr)

        # TODO: Fit the PCA on the scaled training data
        pca = PCA(n_components=ncomp)
        pca.fit(Xtrs)
        Ztr = pca.transform(Xtrs)

        # TODO: Train a classifier on the transformed training data
        # Use a logistic regression classifier
        # logreg = LogisticRegression(multi_class='auto', solver='lbfgs')
        logreg = LogisticRegression(multi_class='auto', solver='lbfgs')
        logreg.fit(Ztr,ytr)

        # TODO: Transform the test data through data scaler and PCA
        Xtss = scaling.transform(Xts)
        Zts = pca.transform(Xtss)

        # TODO: Predict the labels the test data
        yhat = logreg.predict(Zts)

        # TODO: Measure the accuracy
        # acc[icomp, ifold] = ...
        acc[icomp, ifold] = np.mean(yhat == yts)

```

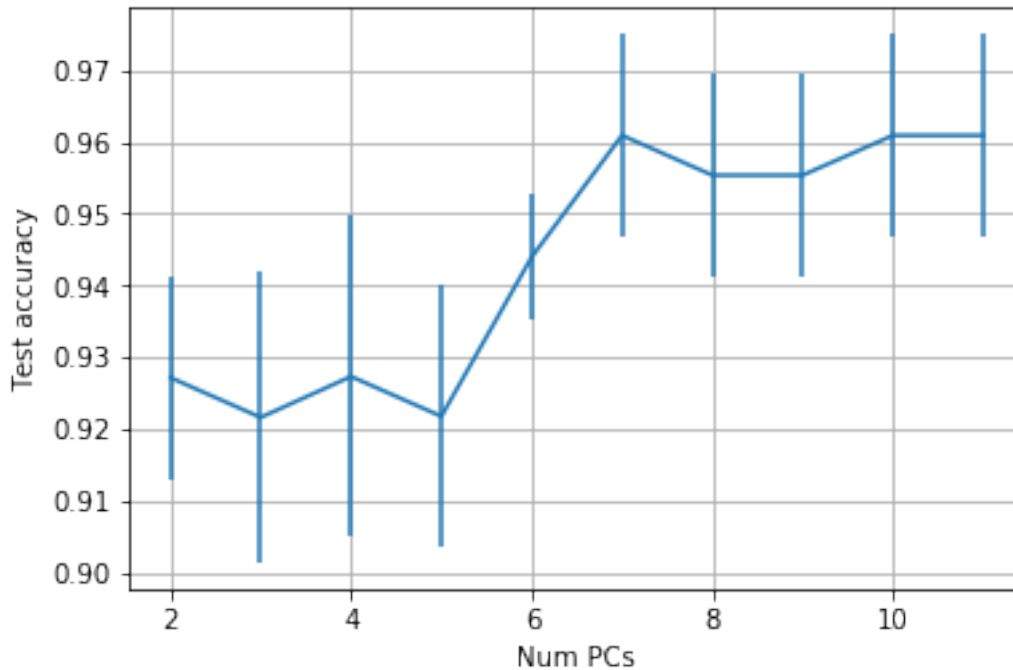
Use the `plt.errorbar` function to plot the mean accuracy with error bars corresponding to the standard error of the accuracy as a function of the number of components. Find the optimal number of PCs to use according to the normal rule and one SE rule.

```
In [9]: # TODO:
        # acc_mean = ...
        # acc_se = ...
        # plt.errorbar(...)
        acc_mean = np.mean(acc, axis=1)
        acc_se = np.std(acc, axis=1) / np.sqrt(nfold-1)
        plt.errorbar(ncomp_test, acc_mean, yerr=acc_se)
        plt.grid()
        plt.xlabel('Num PCs')
        plt.ylabel('Test accuracy')

        # TODO: Optimal order with the normal rule
        im = np.argmax(acc_mean)
        print('Optimal number of PCs (normal rule) = %d' % (ncomp_test[im]))
        print('Best test accuracy = %7.4f' % acc_mean[im])

        # TODO: Optimal order with one SE rule
        acc_tgt = acc_mean[im] - acc_se[im]
        I = np.where(acc_mean >= acc_tgt)[0]
        ise = I[0]
        print('Optimal number of PCs (one SE) = %d' % (ncomp_test[ise]))
        print('Best test accuracy = %7.4f' % acc_mean[ise])
```

```
Optimal number of PCs (normal rule) = 7
Best test accuracy = 0.9610
Optimal number of PCs (one SE) = 7
Best test accuracy = 0.9610
```



1.4 Hyper-Parameter Optimization with GridCV.

We will now try a more complex classifier -- a support vector classifier with a radial basis function. When we use such a classifier, there will be a number of parameters to tune. When the number of parameters to tune becomes large, writing a loop over multiple parameters as we did above becomes cumbersome. The `sklearn` package has a very nice routine, `GridSearchCV` to perform this sort of parameter search.

Before, we do this we need to create an estimator Pipeline. An estimator pipeline is a sequence of transformations followed by an estimator that will operate on the transformed data. Create the following pipeline:

- Create a `StandardScaler()` object called `scaler` for the first transformation
- Create a `PCA()` object called `pca` for the second transformation
- Create a `SVC()` object called `svc` for the final SVM classifier. Set the parameter `kernel='rbf'`.

Once you have the three steps defined, you can create the pipeline with the command:

```
pipe = Pipeline(steps=[('scaler', scaler), ('pca', pca), ('svc', svc)])
```

```
In [10]: from sklearn.pipeline import Pipeline
         from sklearn.model_selection import GridSearchCV
         from sklearn.svm import SVC

         # TODO
         # scaler = ...
```

```

# pca = ...
# svc = ...
# pipe = Pipeline(steps=[('scaler', scaler), ('pca', pca), ('svc', svc)])
scaler = StandardScaler()
pca = PCA()
svc = SVC(kernel='rbf')
pipe = Pipeline(steps=[('scaler', scaler), ('pca', pca), ('svc', svc)])

```

We next define all the parameters that we want to search over. Define the following arrays:

- `ncomp_test`: values from 3 to 10 representing number of PCs to test
- `C_test`: values of `C` in the SVC to test. Use 10^{-2} , 10^{-1} , ... , 10^3
- `gam_test`: values of `gamma` in the SVC to test. Use 10^{-3} , 10^{-2} , ... , 10^1

```

In [11]: # TODO
# ncomp_test = ...
# c_test = ...
# gam_test = ...
ncomp_test = np.arange(3,10)
c_test = np.logspace(-2,3,6)
gam_test = np.logspace(-3,1,5)

```

Next, we create a dictionary `params` of the form:

```
params = {'pca__n_components': ncomp_test, 'svc__C' : c_test, ...}
```

Each key in the dictionary is the of the form `estimator__param` and the value is the values to be tested.

```

In [12]: # TODO
# params = ...
params = {'pca__n_components': ncomp_test, 'svc__C' : c_test, 'svc__gamma': gam_test}

```

Finally, an object `estimator = GridSearchCV(...)` from `pipe` and `params`. Set `cv=5`, `train_score=True` and `iid=False`. Fit the estimator from the data `X,y`. Then the estimator will perform the cross-validation over all the parameters. This may take a minute since we are search over so many parameters.

```

In [13]: # TODO
# estimator = GridSearchCV(...)
# estimator.fit(...)
estimator = GridSearchCV(pipe, params,cv=5,return_train_score=True,iid=False)
estimator.fit(X,y)

```

```

Out[13]: GridSearchCV(cv=5, error_score='raise-deprecating',
                      estimator=Pipeline(memory=None,
                      steps=[('scaler', StandardScaler(copy=True, with_mean=True, with_std=True)), ('p
                      svd_solver='auto', tol=0.0, whiten=False)), ('svc', SVC(C=1.0, cache_size=200, clas
                      decision_function_shape='ovr', degree=3, gamma='auto_deprecated',

```



```

kernel='rbf', max_iter=-1, probability=False, random_state=None,
shrinking=True, tol=0.001, verbose=False))]),
    fit_params=None, iid=False, n_jobs=None,
    param_grid={'pca__n_components': array([3, 4, 5, 6, 7, 8, 9]), 'svc__C': array
    pre_dispatch='2*n_jobs', refit=True, return_train_score=True,
    scoring=None, verbose=0)

```

Print the best test score and best parameters. They are fields in estimator. If you did it correctly, it should be a little higher than the logistic regression (about 0.97 to 0.98 accuracy).

```

In [14]: # TODO
print('Best test accuracy %7.3f' % estimator.best_score_)
print('Best params: %s' % str(estimator.best_params_))

```

```

Best test accuracy    0.978
Best params: {'pca__n_components': 5, 'svc__C': 1.0, 'svc__gamma': 0.1}

```

Finally, you can get the test score for all the parameter choices from

```
test_score = estimator.cv_results_['mean_test_score']
```

Use the `imshow` command to plot the mean test score over gamma and C for the value `n_components=5`.

```

In [16]: test_score = estimator.cv_results_['mean_test_score']

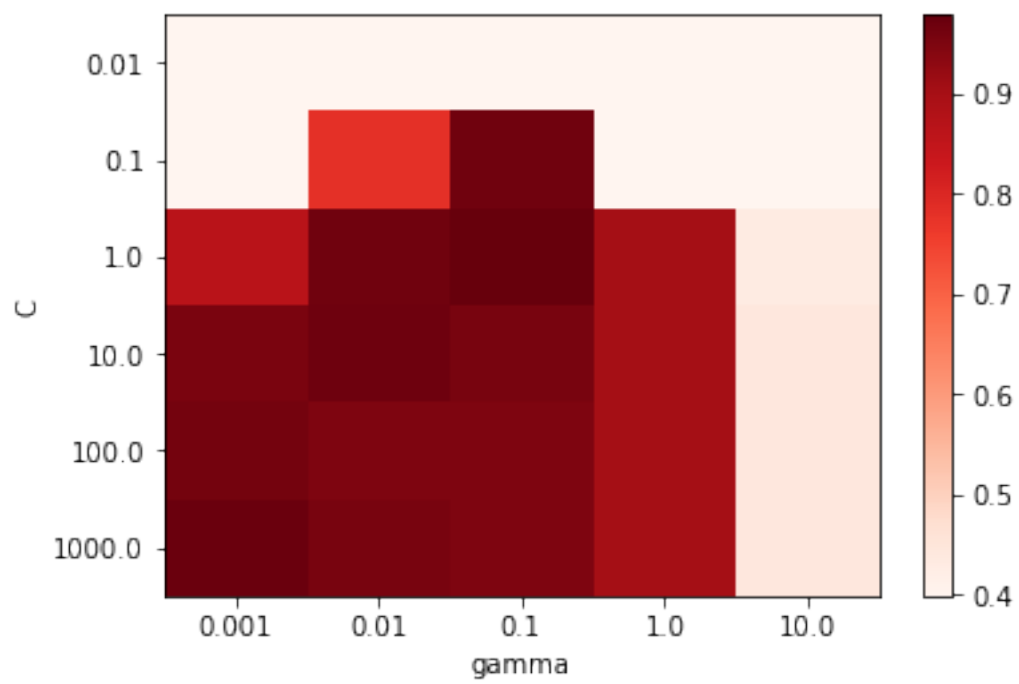
num_nc = len(ncomp_test)
num_c = len(c_test)
num_gam = len(gam_test)
test_score = test_score.reshape((num_nc, num_c, num_gam))

i = np.where(ncomp_test == 5)[0][0]

plt.imshow(test_score[i,:,:], aspect='auto', cmap='Reds')
plt.ylabel('C')
plt.xlabel('gamma')
plt.colorbar()

ax = plt.gca()
ax.set_yticks(np.arange(0,num_c))
ax.set_yticklabels(c_test)
ax.set_xticks(np.arange(0,num_gam))
_ = ax.set_xticklabels(gam_test)

```



In []:

In []: